

SPECIFICATIONS — A2UI Protocol v0.9.1

Implementation & Conformance

Project: Deterministic Design System Manager ("Grace AI") — Raibach Interactive Design Studio Protocol: A2UI v0.9.1 (Agent-to-User Interface), the current production release published by the A2UI project (Google, Apache 2.0) Canonical sources:

- Protocol specification: <https://a2ui.org/specification/v0.9.1-a2ui/>
- Application component catalog \$id: https://raibach.net/a2ui/catalogs/prompt-composer/v0_9_1/catalog.json
- Source of truth for this implementation: `frontend/src/components/A2UI/component-catalog.json`

Status of this document: Authored 2026-08-25 from source review against the running codebase. Every conformance claim below cites its implementing file. Claims marked  were additionally live-verified on 2026-07-27 during the remediation recorded in [READ-ME/A2UI_TRUE_VS_FAKE_AUDIT.md](#). Items not yet built are labeled  SOON rather than omitted — this document describes the specification *and* the honest distance between specification and current build.

Prototype scope statement

This repository is a working prototype demonstrating the A2UI protocol end-to-end. Two categories of "security" must be separated when evaluating it:

1. Protocol-level integrity — in scope, implemented, and audited. The A2UI spec's core safety rule is that an agent sends declarative JSON, never executable code, and that servers generate UI only from a trusted component catalog. Those obligations are enforced in this codebase (see §2 and the conformance matrix) and are the subject of the TRUE-vs-FAKE audit referenced above.
2. Application/infrastructure hardening — deliberately deferred. Authentication is prototype-grade: identity arrives via an `X-User-ID` header with a `DEFAULT_USER_ID` fallback (`backend/`

deps.py::get_user_id_from_header), admin checks are a documented stub (backend/deps.py::user_is_admin, dev mode allows all), and the frontend login is a dev PIN gate. Role resolution reads users.prompt_role from PostgreSQL but there is no real session auth. These are scaffolding for the demo, not production design, and are called out wherever they appear so no reviewer mistakes them for hardened mechanisms.

§1 — Standard validation error format

Any server-side rejection of an invalid component payload uses the spec's standard error envelope. All four fields are required; code MUST be "VALIDATION_FAILED"; path is a JSON Pointer (RFC 6901) to the failing field.



json

```
{
  "error": {
    "code": "VALIDATION_FAILED",
    "surfaceId": "main",
    "path": "/components/0/component",
    "message": "Component 'X' is not in the trusted catalog"
  }
}
```

Implementation: backend/deps.py::validate_a2ui_components() raises FastAPI HTTPException(503) carrying exactly this payload. The server never forwards invalid UI to the client — failure is loud at the boundary.

§2 — Protocol baseline (normative requirements adopted)

The following requirements are taken from the official v0.9.1 outline and are the basis of the conformance matrix in §3.

#	Requirement (per official spec section)	Summary
R1	Envelope message structure	Server→client messages are an array; carries "version" and exactly one operation key: createSurface, updateComponents, updateUI
R2	Catalog identification & compatibility	Catalogs carry both \$id and catalogId; servers must reference the client's understood catalog
R3	Component model / adjacency list	UI is a flat list of component objects for anchoring; anchored by a component with id root; search by id.
R4	Trusted catalog allowlist	Custom catalogs restrict the agent to exactly the components that exist in the application; servers must generate messages conforming to
R5	Data model representation & bindings	Bindings use JSON Pointer paths into the data model (e.g. cards)); Dynamic* types accept literals, arrays, and objects
R6	No executable code	Agents reference named capabilities dynamically; no executable code (no eval, no raw HTML)
R7	Prompt → generate → validate	Servers prompt the LLM with the desired UI + schema + catalog, validate the JSON output, render only if valid, otherwise fail loudly using error
R8	Client→server messages	Clients send action and error messages over transport.
R9	Transport decoupling	The envelope is transport-agnostic (A2A, AG UI, plain HTTP); this implementation uses HTTP + JSON

§3 — Conformance matrix

Legend:  implemented & verified ·  partial ·  specified but not yet built (Phase 3+ roadmap).

Requirement	Implementation (file → mechanism)	Status
R1 Envelope = array of single-operation messages	backend/routes/ai.py — all four as updateComponents, updateDataModel "v0.9.1" (12 live messages across decision, console, composer, s	
R1 deleteSurface emission	Single-surface app (surfaceId: "teardown yet	
R2 \$id ≡ catalogId alignment	Constant A2UI_CATALOG_ID in own \$id (https://raibach.net/a2ui/catalog) used in every createSurface	
R3 Adjacency-list component tree	Assembly prompts require a 'root' Column at top"); LLM der	 (serv
R4 Allowlist enforcement before client delivery	validate_a2ui_components() runs a type or missing id → HTTP 500 startup fail-fast (sys.exit(1) if un currently 28 trusted componen	
R5 JSON Pointer data binding (emission)	Data-bound props emitted as { "/cards"}, prompt-section-editor sec data model delivered via update	 (emis
R5 Binding resolution by a generic renderer	Envelope parser extracts both WritingAreaIndex.tsx), but view co (decision_type/cards/session);	 (Phas

R6 Declarative-only, zero executable code	frontend/src/components/A2UISurface deleted — buttons dispatch de html>/<append-html> innerHTML validated against the registry b	✓
R7 Prompt → generate → valid	/api/ai/assemble-surface injects ro into assembly prompts (tempe fails 503 on empty/invalid LLM rendered shortcut	✓
§1 Standard validation error format	Exact four-field payload, code: components/{i}/id, /components/{i}/c	✓
R8 Client→server action/error m	Buttons/actions are declarative today; they dispatch local	→ (Phase SOON)
R8 Capabilities/metadata exchange	Not wired; role capability exchange role-capabilities REST contract in	→ SOON
R9 Transport decoupling (HTTP binding)	Envelope travels as JSON over HTTP POST; no transport-specific constructs	✓ (HTTP)
Registered client functions (for	Specified in Basic Catalog; no FunctionCall evaluation	→ SOON
Two-way binding / read-write contract	Input components exist; server synchronization of client-side edits is not yet	→ SOON

§4 — Subsystem specifications

4.1 Backend assembly pipeline (the agent host)

- Single unified endpoint: POST /api/ai/assemble-surface (backend/routes/ai.py) is the only surface-assembly path. Intents: render-console, render-composer, render-session:{id}, plus a deterministic unsaved-changes

decision surface that the server assembles itself when client context reports unsaved work — still catalog-validated through the same §1 gate before delivery.

- Model-as-architect: the database supplies raw data only; the LLM must return the component adjacency list. If the model is unreachable or returns non-conforming JSON, the endpoint fails with HTTP 503 — the surface does not render without the AI (no hardcoded fallback UI).
- Fail-fast boot: `backend/deps.py` loads `component-catalog.json` at import time; an unreadable or invalid catalog aborts process startup (`sys.exit(1)`), so the validator can never run against an empty allowlist.
- Role governance before inference: `GET /api/ai/manifest` serves the component manifest filtered by the user's departmental role (`governance`, `ux-design`, `research`, `product`, `basic` — matrix in `backend/role_caps.py`). Filtering happens *before* the LLM call: components outside a role's capability set are never placed in the system prompt. (*Prototype note: role identity rests on the unauthenticated X-User-ID header — see scope statement.*)
- Persistence: sessions/versions in PostgreSQL (`prompt_sessions_api`), semantic embeddings in Zilliz Cloud/Milvus (`milvus_save_version` / `milvus_get_versions`); `POST /api/ai/save-surface` has the AI compile section content into a unified prompt + description + tags before persisting.

4.2 Frontend deterministic shell + AI surface

- Deterministic shell: React app shell (`frontend/src/App.tsx` → `pages/TextAreaIndex.tsx`) always renders; the AI fills named slots inside Lit web components (`ai-surface-sandbox` Shadow DOM viewport with `spinner/console/workspace` slots; `workspace-layout` resizable three-pane host). The slot contract is fixed infrastructure — the AI decides content, never chrome.
- Envelope parser: one assembly entry point (`assembleSurfaceWithAI()`) parses the message array, capturing both `updateComponents` and `updateDataModel`; the active view is inferred from data-model shape (decision payload → decision dialog, cards → console, session → composer). No non-spec routing keys cross the wire.

- Command bridge: chat directives such as `<reassemble-console sort filter/>` parse into `a2ui:console-command` CustomEvents that trigger reassembly — user intent re-enters the same validated pipeline rather than mutating the DOM directly.
- Shared contracts: `frontend/src/shared/` holds the typed surface contract, Zod tag registry (generation source for the manifest), event bus, role caps mirror, and AI orchestrator.

4.3 Component catalog

`frontend/src/components/A2UI/component-catalog.json` — 28 trusted components as of 2026-08-25: base layout/content primitives (`Column`, `Row`, `Text`, `Card`, `Button`, `Image`, `ActionGroup`, `SectionEditor`, `DecisionDialog`, `ConsoleCardGrid`, `CompiledOutput`, `ChatPanel`) plus workspace-specific Lit elements (`workspace-layout`, `prompt-section-editor`, `compiled-output-viewer`, `chat-panel`, `version-trace`, `token-cost-readout`, ...). The count is asserted live in the startup log (✓ A2UI Catalog loaded — N trusted components), so drift between catalog and docs is self-announcing.

4.4 Manifest generation

`frontend/scripts/generate-manifest.mjs` generates `manifest.json` from the live tag registry; `/api/ai/manifest` serves it role-filtered. Honest status: the artifact is absent from the latest committed build, so the endpoint currently serves its documented degradation path (role-filtered static capability lists from `role_caps.py`). Regenerating the artifact restores the registry-derived manifest.

§5 — Honest build-status map

Consistent with the project's audit culture (`README/A2UI_TRUE_VS_FAKE_AUDIT.md`), the distance between this specification and the current build:

Capability	State
------------	-------

Envelope protocol (R1 create/update* messages, versioning)	Live — verified in source and live references (2026-07-27 audit)
Server-side zero-trust validation (S1 error format)	Live (P2-1, P2-8)
No-executable-code posture (eval removal, HTML injection)	Live (P2-4)
Model-as-architect with hard-fail semantics	Live — all three intents
Generic adjacency-list renderer (once component model client)	Not built — Phase 3 target; replace hand-keyed view
JSON Pointer binding resolution / two-way binding	Not built — Phase 3 target
Client→server protocol (action/error messages)	Not built — declarative events + REST extend in for new
deleteSurface, registered functions (for)	Not built
Manifest artifact in build output	Degraded — generator exists; artifact manifest.mjs
Production auth/transport (hardening)	Out of prototype scope (see scope statement)

§6 — Verification methodology

Claims in this document can be reproduced without trusting it:

1. Boot the backend; observe the catalog load assertion and component count on stdout.
2. `curl -s -X POST localhost:$PORT/api/ai/assemble-surface -H 'Content-Type: application/json' -d '{"intent":"render-composer"}'` — inspect the returned array: three messages, version: v0.9.1, catalogId equal to the catalog \$id.
3. Inject an unknown component type through any mock LLM path — confirm HTTP 503 with the §1 error body.
4. Search the frontend for `eval()` and `innerHTML =` assignment from AI input — the remaining hits are the explicit BLOCK guards in `A2UISurfaceContainer.tsx`.

References

- Official A2UI v0.9.1 protocol specification: <https://a2ui.org/specification/v0.9.1-a2ui/>
- A2A extension spec: <https://a2ui.org/specification/v0.9.1-a2ui-extension-specification/> · Evolution guide: <https://a2ui.org/specification/v0.9.1-evolution-guide/> · Basic catalog guide: <https://a2ui.org/specification/v0.9.1-basic-catalog-implementation-guide/>
- [READ-ME/A2UI_TRUE_VS_FAKE_AUDIT.md](#) — live-verified compliance ledger and Phase-2 remediation record
- [A2UI_CARD_CONTRACT.md](#) — Figma→Lit card component data contract
- [CHANGELOG.md](#) — day-by-day release history